

Моделирование бросков монеты: Исследование закона больших чисел

Цель работы: проверить работоспособность закона больших чисел на примере правильной монетки, измеряя частоту орлов, отклонение от теоретического значения и затраченное на выполнение программы время.

Используемое оборудование: мобильная ЭВМ «ASUS» на базе процессора Intel Pentium Silver N5000 с 4-мя Гб оперативной памяти. На ЭВМ используется ЯП Python версии 3.14.5 в среде разработки Visual Studio Code.

Краткая теория: закон больших чисел это принцип в теории вероятностей, согласно которому при многократном повторении одного и того же эксперимента средний результат стремится к теоретическому (ожидаемому) значению. Проверка закона происходила с увеличением количества бросков монетки (N).

Ход выполнения: сделаны несколько функций в коде программы, которые выполняют следующие функции:

1. Функция `flip_coin(n)`, принимающая аргумент `n`, подбрасывает монетку `n`-раз и считает сколько раз выпал орёл, а затем отдаёт число выпавших орлов.

2. Функция `single_experiment(n)`, принимающая аргумент `n`, фиксирует начало запуска `flip_coin(n)`, запускает `flip_coin(n)` `n`-раз, фиксирует конец запуска `flip_coin(n)`, вычисляет затраченное время, вычисляет частоту выпадения орлов (число выпавших орлов / `n`), вычисляет отклонение ($| \text{частота выпадения орлов} - 0,5 |$) и возвращает затраченное время, частоту выпадения орлов и отклонение.

3. Функция `run_series(n_runs=5)`, принимающая аргумент `n_runs`, который по умолчанию равен 5, для каждого количества подбросов монетки (10, 100, 10000, 100000, 1000000) запускает `single_experiment(n)` по 5 раз, выводит результаты и считает средние показатели частоты выпадения орлов, отклонения и затраченного времени.

Графики были построены с помощью библиотеки `matplotlib`.

Исходный код:

```

```
import random
import time
```

```
def flip_coin(n) -> int:
 heads_count = 0
 for i in range(n):
 randint_result = random.randint(0, 1)
 if randint_result == 1:
 heads_count += 1
```

```
return heads_count
```

```
def single_experiment(n) -> tuple:
 start_time = time.perf_counter()
 heads = flip_coin(n)
 end_time = time.perf_counter()
 elapsed_time = end_time - start_time
 heads_frequency = heads / n
 deviation = abs(heads_frequency - 0.5)
 return heads_frequency, deviation, elapsed_time
```

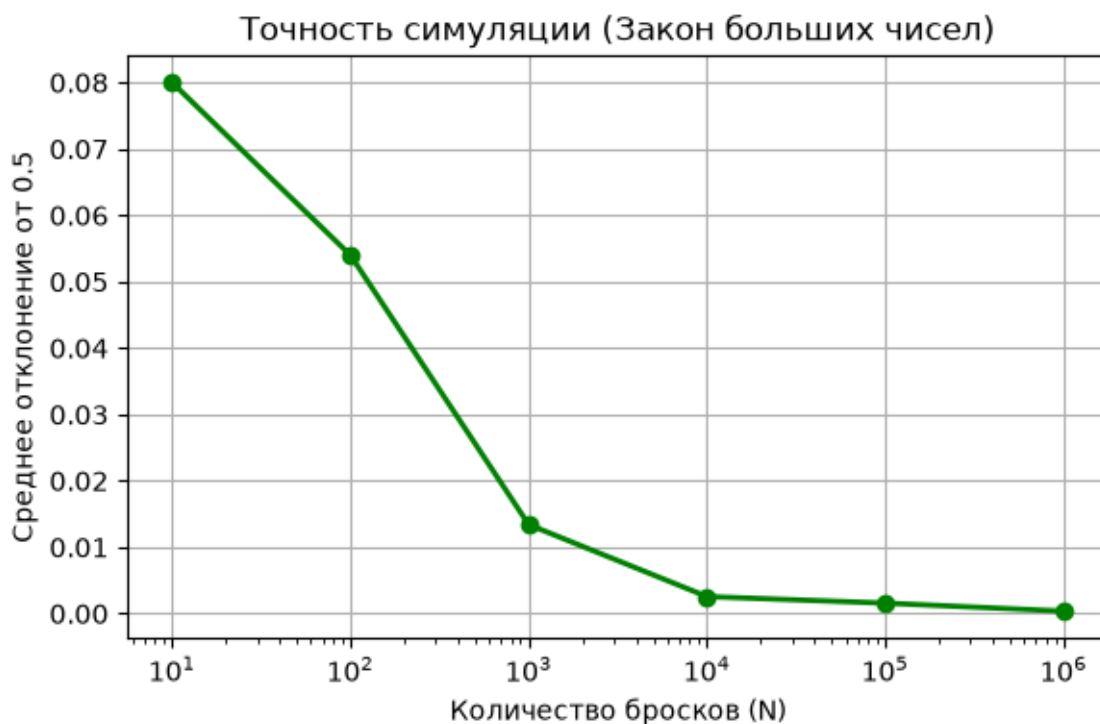
```
def run_series(n_runs=5) -> None:
 flip_nums = [10, 100, 1000, 10000, 100000, 1000000]
 average_deviations = []
 average_elapsed_times = []
 average_head_frequencies = []
 max_time = []
 min_time = []
 for i in flip_nums:
 head_frequency = []
 deviations = []
 elapsed_time = []
 for n in range(n_runs):
 experiment = single_experiment(i)
 print(f"номер {n} эксперимент для {i} бросков: частота орлов
{experiment[0]}; отклонение от 0.5 {experiment[1]}; затраченное время {experiment[2]}")
 head_frequency.append(experiment[0])
 deviations.append(experiment[1])
 elapsed_time.append(experiment[2])
 average_deviations.append(sum(deviations) / len(deviations))
 average_elapsed_times.append(sum(elapsed_time) / len(elapsed_time))
 average_head_frequencies.append(sum(head_frequency) / len(head_frequency))
 max_time.append(max(elapsed_time))
 min_time.append(min(elapsed_time))
 print("-----")
 print("\nСредние показатели для всех бросков:\n")
 for index, i in enumerate(flip_nums):
 print(f"Среднее отклонение для {i} бросков: {average_deviations[index]}")
 print(f"Среднее затраченное время для {i} бросков:
{average_elapsed_times[index]}")
 print(f"Минимальное затраченное время для {i} бросков: {min_time[index]}")
 print(f"Максимальное затраченное время для {i} бросков: {max_time[index]}")
 print(f"Средняя частота орлов для {i} бросков:
{average_head_frequencies[index]}")
 print("-----")
```

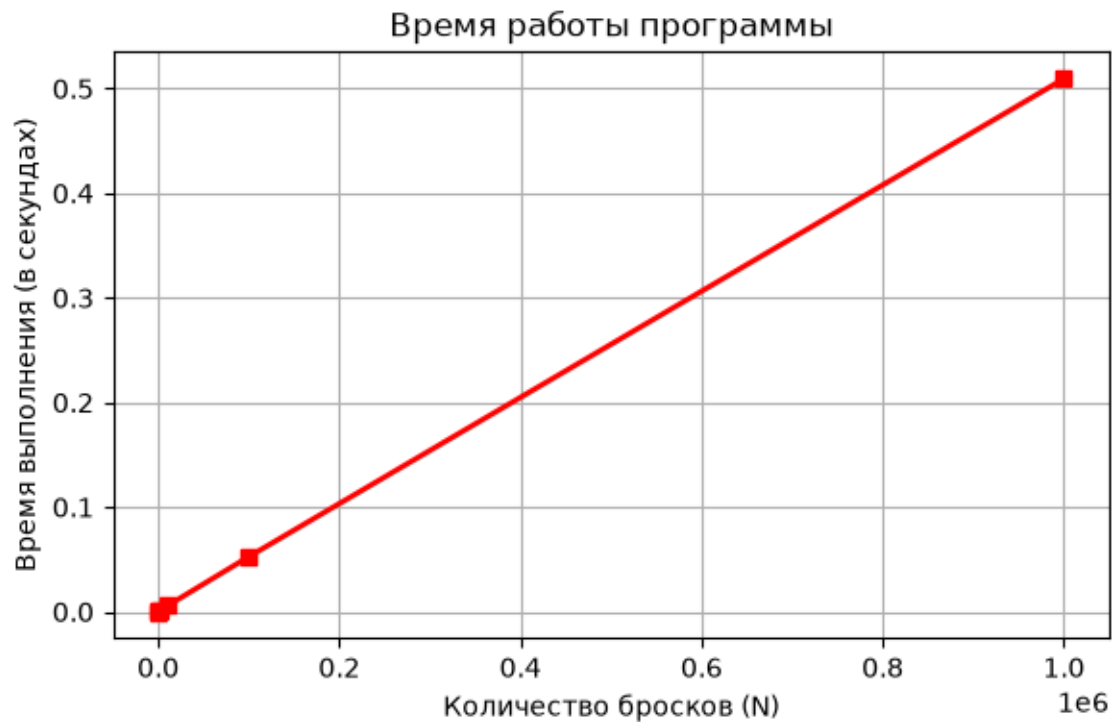
```
run_series()
```

```
...
```

**Результаты эксперимента:**

| Кол-во<br>бросков<br>в | Запуск<br>1<br>(часто<br>та) | Запуск<br>к 2<br>(часто<br>та) | Запуск<br>к 3<br>(часто<br>та) | Запуск<br>к 4<br>(часто<br>та) | Запуск<br>к 5<br>(часто<br>та) | Средн<br>я<br>а      | Средн<br>ее<br>откло<br>нение            | Средн<br>ее<br>время<br>(сек) | Мин.<br>время<br>(сек) | Макс.<br>время<br>(сек) |
|------------------------|------------------------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|----------------------|------------------------------------------|-------------------------------|------------------------|-------------------------|
| 10                     | 0.4                          | 0.4                            | 0.5                            | 0.5                            | 0.3                            | 0.42                 | 0.0799<br>99999                          | 0.0000<br>14140<br>200        | 0.0000<br>07095<br>000 | 0.0000<br>26169<br>000  |
| 100                    | 0.42                         | 0.61                           | 0.54                           | 0.49                           | 0.53                           | 0.518                | 0.0540<br>00000<br>00000<br>00000<br>001 | 0.0000<br>52111<br>1          | 0.0000<br>51154<br>999 | 0.0000<br>54397<br>999  |
| 1000                   | 0.523                        | 0.483                          | 0.488                          | 0.496                          | 0.511                          | 0.5002               | 0.0134<br>00000<br>00000<br>0013         | 0.0005<br>20113<br>6          | 0.0005<br>20113<br>6   | 0.0007<br>46856<br>9    |
| 10000                  | 0.5008                       | 0.4931                         | 0.5044                         | 0.4999                         | 0.4994                         | 0.4995<br>19999<br>9 | 0.0025<br>59999<br>99999<br>99954        | 0.0055<br>59599<br>5          | 0.0051<br>92166<br>0   | 0.0058<br>9042          |
| 10000<br>0             | 0.4991<br>7                  | 0.5026<br>6                    | 0.4992<br>6                    | 0.4977<br>3                    | 0.4985<br>2                    | 0.4994<br>88         | 0.0015<br>75999<br>99999<br>99884        | 0.0525<br>52717<br>2          | 0.0514<br>64767<br>9   | 0.0537<br>75076<br>9    |
| 10000<br>00            | 0.4996<br>78                 | 0.5001<br>2                    | 0.5007<br>6                    | 0.5002<br>68                   | 0.4996<br>39                   | 0.5000<br>93         | 0.0003<br>66200<br>00000<br>00054<br>3   | 0.5093<br>70050<br>9          | 0.5059<br>79379<br>9   | 0.5583<br>07643<br>0    |





Среднее отклонение уменьшается с увеличением  $N$ , что подтверждает работоспособность закона больших чисел.

На примере с 100000 бросками и 1000000 бросков мы можем увидеть, что среднее время работы линейно зависит от количества бросков.

Время запусков в одной и той же серии немного отличается из-за разной нагрузки на ЭВМ и разной температуры процессора ЭВМ.

**Ответы на контрольные вопросы:**

1. Какова теоретическая вероятность выпадения «решки» при броске симметричной монеты? Изменится ли эта вероятность, если предыдущие 5 раз подряд выпадал только «орел»? Объясните почему.

Ответ: теоретическая вероятность выпадения решки равно 0,5.

Вероятность после 5 раз не изменится, так как каждый бросок независим от предыдущего и шансы каждый раз остаются одинаковыми.

2. Что такое закон больших чисел своими словами? Приведите пример его проявления в реальной жизни (не с монетами).

Ответ: закон больших чисел означает, что при множестве повторений реальный результат стремится к теоретическому. Пример из реальной жизни: страховая компания не может предугадать, попадет ли в аварию конкретный водитель в этом году. Но если застраховать миллион водителей, то общая статистика аварий из года в год будет оставаться практически одинаковой и предсказуемой. Это позволяет компании точно рассчитывать стоимость страховки и оставаться в плюсе.

3. Почему для маленького количества бросков (например,  $N = 10$ ) средняя частота орлов сильно колеблется от запуска к запуску, а для большого ( $N = 1000000$ ) она почти всегда равна примерно 0.5?

Ответ: При маленьком количестве бросков, каждый бросок имеет слишком большой вес в средней частоте выпадения орлов. При большом количестве бросков, один бросок имеет слишком маленький вес и почти ничего не собой не решает.

4. Зачем в эксперименте мы делали по 5 запусков для каждого  $N$ , а не ограничились одним запуском? Как это помогает в научных исследованиях?

Ответ: Один запуск может случайно показать аномальный результат, который исказит выводы. Повторение эксперимента по 5 раз для каждого N необходимо для оценки воспроизводимости результатов и уменьшения фактора случайности.

5. Объясните разницу между функцией `random.randint(0, 1)` и функцией `random.random()`. Какую из них удобнее использовать для симуляции броска монеты?

Ответ: Функция `random.randint(0, 1)` возвращает исключительно целые числа: либо 0, либо 1 с равной вероятностью. Функция `random.random()` возвращает случайное вещественное (дробное) число в диапазоне от 0.0 до 1.0 (например, 0.347 или 0.892). Один запуск может случайно показать аномальный результат, который исказит выводы. Повторение эксперимента по 5 раз для каждого N необходимо для оценки воспроизводимости результатов и уменьшения фактора случайности.

**Вывод:** работоспособность закона больших чисел подтвердилась успешно. В ходе работы я научился использовать `matplotlib` на ЯП Python.